

Modular JavaScript

You'll notice quickly that the Node.js community errs towards more modularity rather than less. Case in point, the popular `lodash` JavaScript library is composed of hundreds of useful functions, and each one is available as a separate package if it is all you need. One of the packages for instance is `'lodash.gte'` which is a package composed of a single function that compares two values and tells you if the first is greater than or equal to the second!

It is not uncommon for a single package to depend on dozens of other small packages. To ensure that a single package doesn't bring down your entire running application by breaking something in an update, npm takes version numbers very seriously.

Node.js packages need to follow the SemVer (Semantic Versioning) standard for version numbers that dictates rules for assigning version numbers to your code based on what's changed. A version number has three parts `x.y.z` where `x` is the major version number, `y` the minor and `z` the patch number. Which number should be incremented depends on how the changes made to the code in a package will affect other packages that depend on it.

```

$ npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See 'npm help json' for definitive documentation on these fields
and exactly what they do.

Use 'npm install --save' afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
name: (myproject) myproject
version: (1.0.0) 1.0.0
description: A project to check out the power of Node.js
entry point: (index.js) main.js
test command:
git repository:
keywords:
author: (Rishabh Gupta)
license: (ISC)
  
```

The `npm init` command walks you through creating a basic `package.json` file, and you can take it from there.

For instance, if there is a minor fix to a package that fixes a bug, or clarifies some documentation or otherwise makes changes that have no effect on its public API, then the patch version is to be incremented. A change that adds features, but in a way that doesn't affect existing uses of the API will increment the minor version number. Finally any change that can break existing code using the public API should increment the major version number.

Since version numbers generally dependable, npm allow you to specify what kinds of upgrades you are OK with for which package.

ASIDE: SemVer only has to do with the effect of the changes you make to your package. So if you make a necessary bug fix that unfortunately breaks some existing applications, it should come with a new major version!

*pointers

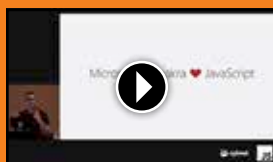
>> Interesting developer news



*Responsive Apps with OpenUI5

>> OpenUI5 is an open source web app development framework by SAP. Watch this video to learn more about developing apps with it.

<http://dgit.in/RaOui>



*Inside Chakra

>> Get an inside look into Microsoft's Chakra JavaScript engine used by Internet Explorer and their latest Edge browser.

<http://dgit.in/InsChkra>



*The Route to HTTP / 2

>> HTTP, one of the central protocols that runs the entire internet is due for an upgrade! Watch this talk to find out what this means and why it's important.

<http://dgit.in/RtHTTP2>



*Electroknit

>> This fascinating talk is about interfacing with legacy hardware using Node.js and how much there is in common between knitting and programming.

<http://dgit.in/Electroknit>



Once you have installed a package, you can import a package to use in your code using `require('package_name')` syntax to access the package functionality, similarly to how you'd access a core library.

If you have multiple JavaScript files in your project you can use this syntax with the path to a file in your project to include that code. For instance `const Button = require('./components/Button')` will load the file `Button.js` in the `components` directory.

Modularising your own Code

When Node.js was being developed, JavaScript will still predominantly a browser-only technology and had no support for modules. Node.js based their module system on the CommonJS standard for JavaScript modules. The `require` function syntax for loading other JavaScript packages is what we have as a result.

To make the JavaScript module system work, any code you write that you intend to import in another file should explicitly export the bits of code that the other files should have access to. An "official" standard module system is now in development for JavaScript which is similar conceptually but with a slightly different syntax.

The Node.js convention for JavaScript modules is that any data or function that you attach to the `module.exports` property can be imported when you use `require` on that file.

Let's create a simple file to demonstrate all of this:

```

const PI = 3.14159;
exports.add = function(x, y) {
  return x + y;
}
// In the following we use the new
// arrow function syntax for brevity
exports.multiply = (x, y) => x * y;
exports.circleArea = (r) => PI * r * r;
  
```

Let's say this code is in a file called `math.js` in the same directory as your main code. You could then import this using: `const math = require('./math')`; Note that the `.js` extension is not required.

Now the functions defined in the above code will be available under the `math` object. So you could run them as `math.add(1, 3)` or `math.circleArea(4)`. The `PI` constant will not be available however, as it is not explicitly attached to `export`.

If a directory contains a `index.js` file then you can use `require` on that directory itself. This `index.js` file can then import other files from that directory allowing better organisation and encapsulation of your code. In the case of packages (i.e. folders with a `package.json` file) you can specify which file should be imported when that package the "main" field in the `package.json` file.